AI ENGINEER TRACK · DAY 2 · ≈ 2 HOURS

Prompting & Structured Output that downstream code can trust

Prompt structure · JSON mode vs Structured Outputs · Pydantic · Function calling | Project: extract HAL financials

GOAL

Day 2 goal: Turn free-text LLM output into validated, typed data: write production-grade prompts, use Structured Outputs with Pydantic, verify every extracted value against its source, and run one function-calling loop.

Why it matters: In real systems an LLM is a component whose output feeds code — a database, an API, an agent's next step. If the output isn't a guaranteed shape, the pipeline breaks. Every 50+ LPA loop asks *"how do you make LLM output reliable?"* and *"JSON mode vs function calling?"*. Structured output is also the base of tool calling, which is the base of agents.

Time plan: concepts 40 min · hands-on 60 min · interview Q&A out loud 20 min

1 Concepts — only what production uses

A. Anatomy of a production prompt

Block	What goes in it	Why
System message	Role, hard rules, what to do when information is missing, output constraints	Highest-priority instructions; stable across calls → put first so prompt caching hits
Context (delimited)	Retrieved text inside clear tags, e.g. `<context>…</context>`, with source markers like `[PAGE 12]`	Separates data from instructions (also a first line of prompt-injection defence); markers enable citations
Task	One specific instruction in the user message	One task per call is more accurate and easier to evaluate than one mega-prompt
Output contract	Enforced by a schema (Structured Outputs), not by prose like "return JSON"	Prose format instructions fail a small % of the time — at scale that is thousands of broken records

B. Prompt patterns that actually move quality

- **Explicit escape hatch:** "If the value is not stated in the context, return null." Without it, the model fills gaps with plausible numbers. Make schema fields **nullable** so null is a legal answer.
- **Few-shot examples:** 1–3 input→output examples when the format or judgement is subtle (e.g. which line counts as "revenue"). Cost: extra input tokens on every call. Risk: the model copies values from examples — use clearly fake data.
- **Reasoning before answer:** the model generates JSON fields **in schema order**. Put `evidence` / `reasoning` fields **before** the final value so the value is conditioned on the quote. (Reasoning models do this internally; for standard models, field order matters.)
- **Decompose:** extract → validate → compute in separate steps. Never let the LLM do arithmetic you can do in code.
- **Prompts are code:** store in files under git, give each a version id, log the version with every call, and run your eval set before changing one. A prompt edit is a deployment.

C. Four ways to get structured output — know when to use each

Method	Guarantee	Use when
Prompt only ("reply in JSON")	None — may add markdown fences, prose, missing keys	Never in production
JSON mode `{"type":"json_object"}`	Valid JSON syntax only; **no** schema guarantee. OpenAI requires the word "JSON" in the messages	Legacy / models without schema support
Structured Outputs `json_schema, strict: true`	Output **conforms to your schema** (constrained decoding: tokens that would break the schema are blocked)	**Default for extraction, classification, any data the code consumes**
Function / tool calling	Model returns a tool name + arguments matching the tool's schema (use `strict: true`); **your code** executes it	The model must **decide an action** — which tool, whether to call one at all. Basis of agents

Rule of thumb: **Structured Outputs to shape the answer; tool calling to let the model choose an action.** Azure OpenAI, Anthropic (tool use) and Gemini offer equivalent features; the concept transfers.

D. Pydantic as the contract

- Define the schema once as a Pydantic model → the SDK converts it to JSON Schema for the API and parses the reply back into a typed object (`client.chat.completions.parse(..., response_format=Model)`).
- `Field(description=...)` is sent to the model — **descriptions are part of the prompt**. Write them precisely (units, format, what null means).
- Strict-mode constraints: every field must be **required** (express "optional" as `Optional[X]` = nullable, no default), no extra keys, and only a subset of JSON Schema keywords is supported. Use `Literal[...]` for enums.
- **Schema-valid ≠ correct.** The schema guarantees shape, not truth. Business rules (value appears on the cited page, PAT < revenue, fiscal year matches the report) are checked **in your code after parsing**.

E. The reliability loop (what production code looks like)

```
Validate → feedback → retry → quarantine

call LLM (strict schema, temperature 0)
  ├─ message.refusal set?        → stop, log, return error  (safety refusal, don't retry blindly)
  ├─ finish_reason == "length"?  → truncated (SDK parse() raises LengthFinishReasonError)
  ├─ parse into Pydantic         → structural guarantee
  └─ business validation         → errors? send them back to the model as feedback, retry (max 2)
                                    → still failing? quarantine record + alert   (never write bad data)
```

F. Function calling — the exact message flow

- 1) You send messages + `tools` (name, description, JSON-schema parameters). 2) Model replies with `finish_reason="tool_calls"` and `tool_calls=[{id, name, arguments}]` — it does **not** run anything. 3) Your code validates the arguments and executes the function. 4) You append the assistant message and a `{"role":"tool", "tool_call_id": id, "content": result}` message. 5) Call the model again; it answers using the result (or calls another tool).
- `tool_choice`: `auto` (model decides), `required` (must call one), or force a specific tool. Models can emit **parallel** tool calls in one turn — handle a list, not one call.
- Tool **descriptions** decide tool selection. Vague descriptions = wrong tool picked. Arguments are untrusted input: validate before executing.

> **Your DE edge:** treat LLM output exactly like an untrusted upstream feed — schema contract, validation rules, a quarantine (dead-letter) table for failed records, and a metric for validation-failure rate per prompt version. Saying this in an interview immediately separates you from candidates who only "call the API and parse the JSON".

2 Hands-on (≈ 60 min) — extract HAL financials into a validated schema

Adds structured extraction to FinDoc RAG. Retrieval is a simple keyword page filter today (real retrieval comes with embeddings). Output: a JSON snapshot where **every number carries its page and a verbatim quote, verified in code**.

Step 0 · Files (5 min)

```
findoc-rag/
├── src/llm/client.py           # Day 1 (unchanged)
├── src/llm/structured.py       # extract() with validate → feedback → retry   ← today
├── src/schemas/financials.py   # Pydantic contract                            ← today
├── scripts/day02_extract.py    # HAL extraction run                           ← today
├── scripts/day02_tools.py      # function-calling loop                        ← today
└── data/processed/             # validated JSON output

pip install --upgrade openai pydantic
touch src/schemas/__init__.py   # after creating the folder
```

Step 1 · The contract (10 min)

```
# src/schemas/financials.py
from typing import Literal, Optional
from pydantic import BaseModel, Field

class Metric(BaseModel):
    # field ORDER matters: quote first, then page, then value -> value is grounded in the quote
    evidence: Optional[str] = Field(description="Verbatim quote (max 25 words) from the cited page that "
                                                "contains the number. null if not found.")
    source_page: Optional[int] = Field(description="Page number from the [PAGE n] marker. "
                                                   "null if not found.")
    value: Optional[float] = Field(description="Number exactly as printed, commas removed, in the snapshot "
                                               "unit. Do not convert or compute. null if not found.")

class FinancialSnapshot(BaseModel):
    company: str
    fiscal_year: str = Field(description="Fiscal year the figures belong to, format FY2025-26")
    currency: Literal["INR", "USD"]
    unit: Literal["crore", "lakh", "million", "billion"] = Field(
        description="Unit the report states for these figures")
    basis: Literal["standalone", "consolidated", "unknown"]
    revenue_from_operations: Metric
    profit_after_tax: Metric
    order_book: Metric

METRICS = ["revenue_from_operations", "profit_after_tax", "order_book"]
```

Step 2 · extract() — the reliability loop (20 min)

```
# src/llm/structured.py
import time, uuid
from openai import LengthFinishReasonError
from pydantic import ValidationError
from src.llm.client import CLIENT, CFG, _cost, _log

class ExtractionError(Exception): ...

def extract(messages, schema, validate=None, model=None, max_attempts=3,
            max_tokens=1500, tag="", prompt_version=""):
    """Structured Outputs + business validation + feedback retry. Returns a schema instance."""
    model = model or CFG["default_model"]
    msgs = list(messages)
    for attempt in range(1, max_attempts + 1):
        t0 = time.perf_counter()
        try:
            resp = CLIENT.chat.completions.parse(      # older SDK: CLIENT.beta.chat.completions.parse
                model=model, messages=msgs, response_format=schema,
                temperature=0, max_tokens=max_tokens)
        except LengthFinishReasonError:                # parse() raises this on finish_reason="length"
            raise ExtractionError("output truncated - raise max_tokens or shrink the input")
        except ValidationError as e:                   # rare with strict mode, but handle it
            msgs.append({"role": "user", "content": f"Output failed schema validation: {e}. Fix it."})
            continue
        choice, u = resp.choices[0], resp.usage
        _log({"id": str(uuid.uuid4()), "ts": time.time(), "tag": tag, "model": model,
              "prompt_version": prompt_version, "attempt": attempt,
              "prompt_tokens": u.prompt_tokens, "completion_tokens": u.completion_tokens,
              "total_s": round(time.perf_counter() - t0, 3), "finish_reason": choice.finish_reason,
              "cost_usd": round(_cost(model, u.prompt_tokens, u.completion_tokens), 6)})

        if choice.message.refusal:                     # safety refusal: content is None
            raise ExtractionError(f"model refused: {choice.message.refusal}")

        obj = choice.message.parsed                    # typed Pydantic object
        errors = validate(obj) if validate else []
        if not errors:
            return obj
        print(f"[attempt {attempt}] validation errors:", errors)
        msgs.append({"role": "assistant", "content": choice.message.content})
        feedback = ("Your output failed these checks:\n- " + "\n- ".join(errors)
                    + "\nReturn corrected output. Use null if a value is not stated.")
        msgs.append({"role": "user", "content": feedback})
    raise ExtractionError(f"failed after {max_attempts} attempts")
```

Step 3 · Run the extraction (15 min)

```
# scripts/day02_extract.py     run:  python -m scripts.day02_extract data/raw/HAL_AR-2025-26.pdf
import sys, re, json
from pathlib import Path
from pypdf import PdfReader
from src.schemas.financials import FinancialSnapshot, METRICS
from src.llm.structured import extract, ExtractionError

PROMPT_VERSION = "extract_financials.v1"
SYSTEM = """You are a financial data extraction engine.
Rules:
- Use ONLY the text inside <context>. Never use prior knowledge.
- If a value is not explicitly stated, return null for evidence, source_page and value.
- Copy numbers exactly as printed (commas removed) in the unit the report uses. Do not convert or compute.
- evidence must be copied verbatim from the cited page.
- Prefer standalone figures; if only consolidated are given, use them and set basis accordingly."""

pages = [p.extract_text() or "" for p in PdfReader(sys.argv[1]).pages]
norm = lambda s: re.sub(r"\s+", " ", s).lower()

# 1) naive retrieval: pages with the most keyword hits (replaced by vector search on later days)
KEYS = ["revenue from operations", "profit after tax", "order book"]
scores = sorted(((sum(norm(p).count(k) for k in KEYS), i) for i, p in enumerate(pages)), reverse=True)
top = sorted(i for s, i in scores[:4] if s > 0)
context = "\n\n".join(f"[PAGE {i + 1}]\n{pages[i][:6000]}" for i in top)
print("pages sent:", [i + 1 for i in top])

# 2) business validation = grounding check in code, not trust
def grounding_errors(snap):
    errs = []
    for name in METRICS:
        m = getattr(snap, name)
        if m.value is None:
            continue
        if m.source_page is None or m.source_page - 1 not in top:
            errs.append(f"{name}: source_page {m.source_page} was not in the context"); continue
        page = norm(pages[m.source_page - 1])
        if not m.evidence or norm(m.evidence) not in page:
            errs.append(f"{name}: evidence is not a verbatim quote from page {m.source_page}")
        candidates = {f"{m.value:.2f}", f"{m.value:.1f}", f"{m.value:g}"}
        if not any(c in page.replace(",", "") for c in candidates):
            errs.append(f"{name}: value {m.value} does not appear on page {m.source_page}")
    r, p = snap.revenue_from_operations.value, snap.profit_after_tax.value
    if r and p and p >= r:
        errs.append("profit_after_tax must be smaller than revenue_from_operations")
    return errs

messages = [{"role": "system", "content": SYSTEM},
            {"role": "user", "content": f"<context>\n{context}\n</context>\n\n"
                                        "Extract the financial snapshot for the latest "
                                        "fiscal year in this report."}]
try:
    snap = extract(messages, FinancialSnapshot, validate=grounding_errors,
                   tag="day02_extract", prompt_version=PROMPT_VERSION)
except ExtractionError as e:
    Path("data/quarantine").mkdir(parents=True, exist_ok=True)
    Path("data/quarantine/hal_snapshot.err").write_text(str(e)); sys.exit(f"quarantined: {e}")

Path("data/processed").mkdir(parents=True, exist_ok=True)
Path("data/processed/hal_snapshot.json").write_text(snap.model_dump_json(indent=2))
print(snap.model_dump_json(indent=2))
```

Step 4 · Function-calling loop (10 min)

```
# scripts/day02_tools.py     run:  python -m scripts.day02_tools
import json
from src.llm.client import CLIENT

snap = json.load(open("data/processed/hal_snapshot.json"))

def compute_ratio(numerator: float, denominator: float) -> float:
    return round(numerator / denominator * 100, 2)

TOOLS = [{"type": "function", "function": {
    "name": "compute_ratio",
    "description": "Returns numerator / denominator * 100 rounded to 2 decimals. "
                   "Use for ANY margin or percentage. Never compute percentages yourself.",
    "strict": True,
    "parameters": {"type": "object",
                   "properties": {"numerator": {"type": "number"}, "denominator": {"type": "number"}},
                   "required": ["numerator", "denominator"], "additionalProperties": False}}}]
REGISTRY = {"compute_ratio": compute_ratio}

msgs = [{"role": "system", "content": "Answer only from the given data. Use tools for arithmetic."},
        {"role": "user", "content": f"Data: {json.dumps(snap)}\nWhat is the net profit margin?"}]

for _ in range(3):                                          # bounded loop - never while True
    r = CLIENT.chat.completions.create(model="gpt-4o-mini", messages=msgs, tools=TOOLS, temperature=0)
    msg, finish = r.choices[0].message, r.choices[0].finish_reason
    print("finish_reason:", finish)
    if finish != "tool_calls":
        print("ANSWER:", msg.content); break
    msgs.append(msg)                                        # assistant turn that requested the tools
    for tc in msg.tool_calls:                               # may be several (parallel calls)
        args = json.loads(tc.function.arguments)
        result = REGISTRY[tc.function.name](**args)         # YOUR code executes, not the model
        print(f"tool: {tc.function.name}({args}) -> {result}")
        msgs.append({"role": "tool", "tool_call_id": tc.id, "content": json.dumps({"result": result})})
```

DELIVERABLE

Record these in your notes:

- Which pages were sent, and the extracted revenue, PAT and order book with page numbers — check them against the PDF yourself.
- Did any grounding check fail on attempt 1? What error was fed back, and did attempt 2 fix it?
- Tokens and cost per extraction (from `logs/llm_calls.jsonl`, now tagged with `prompt_version`).
- **Break it on purpose:** change `value: Optional[float]` to `value: float` and ask for a metric that isn't in the context (e.g. add `ebitda: Metric`, remove its pages). Watch the model invent a number. Revert.
- In the tools run: the arguments the model passed, and confirm the final margin equals your code's result.

3 Top interview questions — crisp model answers

Q1. JSON mode vs Structured Outputs vs function calling — when do you use each?

JSON mode only guarantees syntactically valid JSON, not my schema, so I don't rely on it. Structured Outputs with **strict** schema constrains decoding so the output always matches the schema — my default for extraction and classification. Function calling is for when the model must **decide an action**: which tool, with what arguments, or none. The model only proposes the call; my code validates and executes it.

Q2. How do you make LLM output reliable enough for a downstream system?

Layers. (1) Strict schema so shape is guaranteed. (2) Pydantic parsing into typed objects. (3) Business validation in code — ranges, cross-field rules, and grounding checks that each value appears in the cited source. (4) On failure, feed the exact errors back and retry, bounded to 2. (5) Still failing → quarantine the record and alert, never write it. I track validation-failure rate per prompt version as a quality metric — same as data-quality checks in a pipeline.

Q3. How do you reduce hallucination in document extraction?

Restrict the model to delimited context, make every field nullable with an explicit “return null if not stated” rule, require a verbatim evidence quote and page per value, then verify the quote and number exist on that page in code. Temperature 0, one task per call, and no arithmetic by the LLM — computed fields are done in code or via tools.

Q4. Does the model execute the function in function calling? Walk me through the flow.

No. I send tools with JSON-schema parameters. The model returns `finish_reason=tool_calls` with tool name and arguments. My code validates the arguments, runs the function, and appends a tool message with the matching `tool_call_id`. I call the model again so it can answer or call another tool. I handle parallel calls, cap the loop, and treat arguments as untrusted input.

Q5. In production you suddenly see truncated or unparseable JSON. What are the likely causes?

Most often `finish_reason=length` — output hit max_tokens, usually because input grew or the schema has large lists. Others: strict mode not enabled (JSON mode or prompt-only), a refusal returned instead of content, or a model/version change. Fix: check finish_reason and refusal on every call, size max_tokens for the worst case, paginate long lists, and pin model versions.

Q6. Does the order of fields in your schema matter?

Yes. Models generate fields in schema order, so a field can only be conditioned on fields before it. I put evidence or reasoning fields before the final value or label. That measurably improves grounding for standard models; reasoning models think internally first, so it matters less there.

Q7. How do you manage prompts in production?

As code: versioned files in git, a version id logged with every call, templates with static instructions first for cache hits and dynamic context last. Any change runs against a golden evaluation set before release, and I can roll back by version. Tools like LangSmith help store versions and compare runs.

Q8. When would you use few-shot examples, and what's the downside?

When the format or judgement is subtle and instructions alone aren't enough — e.g. which line item counts as revenue. Downsides: extra input tokens on every call, and the model may copy example values or over-fit to their style. I use 1–3 examples with clearly fake data, and if I need many examples, that's a signal to consider fine-tuning.

4 Common mistakes & production gotchas

Mistake	What happens	Do this instead
“Return JSON” in the prompt only	Markdown fences, extra prose, missing keys in a small % of calls	Structured Outputs with strict schema
JSON mode without the word “JSON” in messages	OpenAI rejects the request	Mention JSON in the prompt — or use Structured Outputs
Non-nullable fields for data that may be absent	Model invents plausible values to fill the field	`Optional[...]` + explicit null rule
Final value before evidence in the schema	Value isn't conditioned on the quote; weaker grounding	Evidence / reasoning fields first
Treating schema-valid as correct	Wrong numbers flow downstream silently	Business + grounding validation in code
Not checking refusal / finish_reason	Crashes on None or half-written output	Check both on every call
Unbounded retry or tool loop	Runaway cost, stuck requests	Cap attempts; quarantine on final failure
LLM does arithmetic	Small numeric errors that look right	Compute in code or via a tool
Vague tool descriptions	Wrong tool or wrong arguments	Describe when to use it, units and constraints
Editing prompts with no version or eval	Silent quality regression in production	Version + golden-set eval before release
Huge nested schemas	Higher latency; first call with a new schema is slower	Split into smaller extractions

5 Done-checklist — tick each item before moving on

- [ ] I can name the four blocks of a production prompt and why context goes inside delimiters with source markers.
- [ ] I can explain the difference between prompt-only JSON, JSON mode, Structured Outputs and function calling, and pick one for a given task.
- [ ] I can explain why fields should be nullable and why field order in the schema matters.
- [ ] I can describe the validate → feedback → retry → quarantine loop and why schema-valid ≠ correct.
- [ ] I can walk through the function-calling message flow, including tool_call_id and parallel calls.
- [ ] `src/schemas/financials.py` and `src/llm/structured.py` written and working.
- [ ] `data/processed/hal_snapshot.json` exists; I checked each value against the PDF page cited.
- [ ] I triggered at least one grounding-check failure (or the break-it test) and saw the feedback retry or the invented value.
- [ ] `day02_tools.py` ran: the model requested `compute_ratio`, my code executed it, and the final answer used the result.
- [ ] Log entries now include `prompt_version` and `attempt`.
- [ ] Answered all 8 interview questions aloud without reading the answers.